

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 10 | Part 1

Backtracking and Dynamic Programming

Beyond Greedy

- ▶ Greedy algorithms are typically **fast**, but may not find the optimal answer.
- ▶ Brute force guarantees the optimal answer, but is **slow**.
- ▶ Can we guarantee the optimal answer and be faster than brute force?

Today

- ▶ The **backtracking** idea.
- ▶ It is a useful, general algorithm design technique.
- ▶ And the foundation of **dynamic programming**.

CS-GY 6033

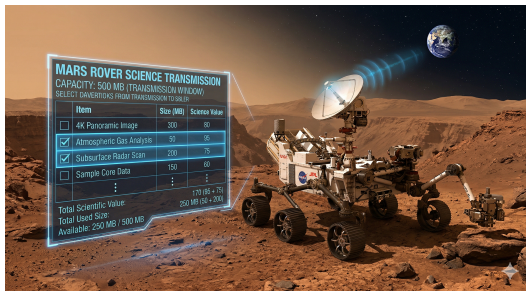
Design and Analysis of Algorithms I

Lecture 10 | Part 2

The 0-1 Knapsack Problem

0-1 Knapsack

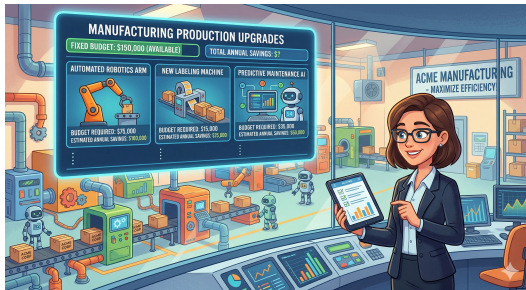
- ▶ Suppose you are a flight controller for a Mars Rover.
- ▶ The rover has a limited transmission window with a total capacity of 500 MB.
- ▶ You have a list of n scientific data packages ready to be sent to Earth.



- ▶ **Goal:** Maximize the total scientific value of the data transmitted within the 500 MB limit.

0-1 Knapsack

- ▶ You are a product portfolio manager at a manufacturing corporation.
- ▶ Fixed budget of \$150,000 allocated for a new round of production line upgrades.
- ▶ A list of n competing efficiency improvement projects you can invest in.



- ▶ **Goal:** Maximize the total annual savings generated by the funded projects within the \$150,000 budget.

0-1 Knapsack

- ▶ Suppose you're a thief.
- ▶ You have a knapsack (bag) that can fit 100L.
- ▶ And a list of n things to possibly steal.



- ▶ **Goal:** maximize total value of items you can fit in your knapsack.

Example

item	size (L)	price
1	50	\$40
2	10	\$25
3	80	\$100
4	5	\$10
5	20	\$20
6	30	\$6
7	8	\$32
8	17	\$34

In the bag: _____

Total value: _____

Space remaining: _____

Greedy

- ▶ Does a greedy approach find the optimal?
- ▶ What do we mean by “greedy”?
- ▶ Idea #1: take most expensive available that will fit.

Example

item	size (L)	price
1	50	\$40
2	10	\$25
3	80	\$100
4	5	\$10
5	20	\$20
6	30	\$6
7	8	\$32
8	17	\$34

In the bag: _____

Total value: _____

Space remaining: _____

Greedy, Idea #2

- ▶ We want items with high value for their size.
- ▶ Define “price density” = $\text{item.price} / \text{item.size}$
- ▶ Idea #2: take item with highest price density.

Example

item	size (L)	price	PD
1	50	\$40	0.80
2	10	\$25	2.50
3	80	\$100	1.25
4	5	\$10	2.00
5	20	\$20	1.00
6	30	\$6	0.20
7	8	\$32	4.00
8	17	\$34	2.00

In the bag: _____

Total value: _____

Space remaining: _____

Greedy is **Not Optimal**

- ▶ Claim: the best possible total value is \$157.
 - ▶ Items 2, 3, and 7.

Never Looking Back

- ▶ Once greedy makes a decision, it never looks back.
- ▶ This is why it may be suboptimal.
- ▶ **Backtracking**: go back to reconsider every previous decision.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 10 | Part 3

Backtracking

Backtracking

- ▶ Reconsider every decision.
- ▶ If we initially tried including x , also try *not* including x .
 - ▶ Find the best solution among those that **include** x
 - ▶ Find the best solution among those that **exclude** x
 - ▶ Return the better of the two.

Backtracking

```
def knapsack(items, bag_size):  
    # choose item arbitrarily from those that fit in bag  
    x = items.arbitrary_item(fitting_in=bag_size)  
  
    # if None, it means there was no item that fit  
    if x is None:  
        return 0  
  
    # assume x should be in bag, see what we get  
    best_with = ...  
  
    # backtrack: now assume x should not be in bag, see what we get  
    best_without = ...  
  
    return max(best_with, best_without)
```

Recursive Subproblems

- ▶ What is `BEST(items, bag_size)` if we assume that `x` **is** in the bag?
- ▶ The best outcome is `x.price` + best choice of remaining items.
- ▶ Imagine choosing `x`.
 - ▶ Your current total value is `x.price`.
 - ▶ You have `bag_size - x.size` space left.
 - ▶ Items left to choose from: `items - x`.
- ▶ Answer: `x.price + BEST(items - x, bag_size - x.size)`

Recursive Subproblems

- ▶ What is $\text{BEST}(\text{items}, \text{bag_size})$ if we assume that x **is not** the bag?
- ▶ Clearly, you want the best outcome for remaining items.
- ▶ Imagine deciding x is not in the bag.
 - ▶ Your current total value is $\odot$.
 - ▶ You have bag_size space left.
 - ▶ Items left to choose from: $\text{items} - x$.
- ▶ Answer: $\odot + \text{BEST}(\text{items} - x, \text{bag_size})$

Backtracking

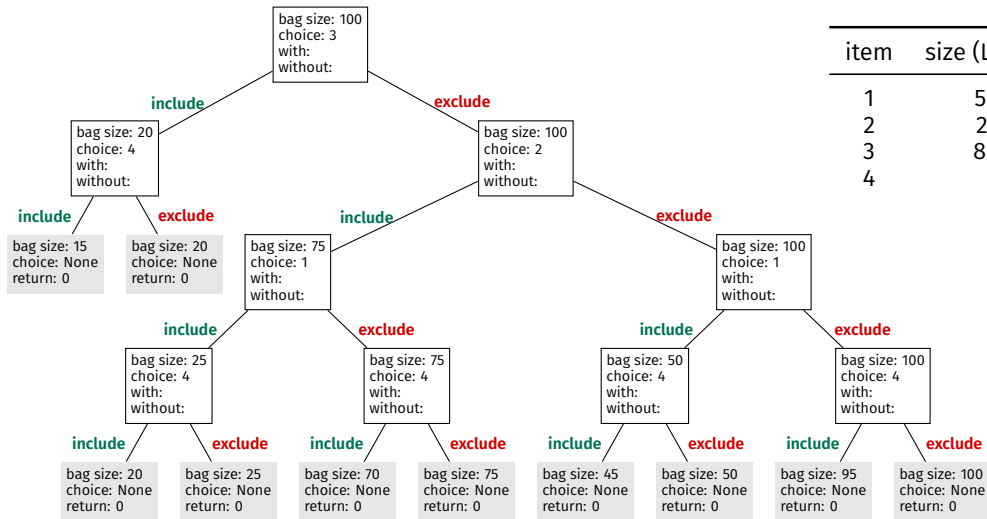
```
def knapsack(items, bag_size):  
    # choose item arbitrarily from those that fit in bag  
    x = items.arbitrary_item(fitting_in=bag_size)  
  
    # if None, it means there was no item that fit  
    if x is None:  
        return 0  
  
    # assume x is in the bag, see what we get  
    best_with = x.price + knapsack(items - x, bag_size - x.size)  
  
    # now assume x is not in bag, see what we get  
    best_without = 0 + knapsack(items - x, bag_size)  
  
    return max(best_with, best_without)
```

Backtracking

```
def knapsack(items, bag_size):  
    # choose item arbitrarily from those that fit in bag  
    x = items.arbitrary_item(fitting_in=bag_size)  
  
    # if None, it means there was no item that fit  
    if x is None:  
        return 0  
  
    items.remove(x)  
    best_with = x.price + knapsack(items, bag_size - x.size)  
    best_without = knapsack(items, bag_size)  
    items.replace(x)  
  
    return max(best_with, best_without)
```

Backtracking

- ▶ **Backtracking**: go back to reconsider every previous decision.
- ▶ Searches the whole tree.
- ▶ Can be thought of as a DFS on implicit tree.



item	size (L)	price
1	50	\$40
2	25	\$25
3	80	\$100
4	5	\$10

Exercise

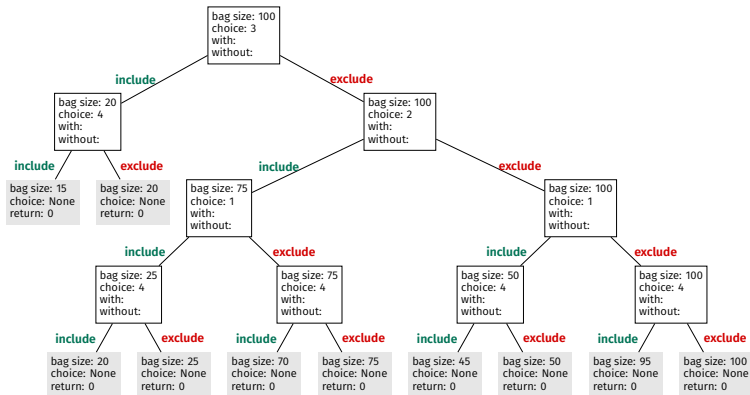
Is the backtracking solution **guaranteed** to find an optimal solution?

Yes!

- ▶ It tries every **valid** combination and keeps the best.
 - ▶ A combination of items is valid if they fit in the bag together.

Leaf Nodes

- Each leaf node is a different valid combination.



Exercise

Suppose instead of choosing an **arbitrary** node we choose most **expensive**. Is it still **guaranteed** to find an optimal solution?

Yes!

- ▶ The choice of node is **arbitrary**.
- ▶ Call tree will change, but all valid combinations are still tried.

Exercise

How does backtracking relate to the greedy approach?
How would you change the code to make it greedy?

Summary

```
def knapsack_greedy(items, bag_size):  
    # choose greedily  
    x = items.most_valuable_item(fitting_in=bag_size)  
  
    # if None, it means there was no item that fit  
    if x is None:  
        return 0  
  
    # assume x is in the bag, see what we get  
    best_with = x.price + knapsack(items - x, bag_size - x.size)  
  
    # in the greedy approach, we don't do this  
    # best_without = knapsack(items - x, bag_size)  
  
    return best_with
```

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 10 | Part 4

Efficiency Analysis

A Benchmark

- ▶ Brute force: try every **possible** combination of items.
 - ▶ Even the **invalid** ones whose total size is too big.
 - ▶ Why? Hard to know which are invalid without trying them.
- ▶ There are $\Theta(2^n)$ possible combinations.
- ▶ So brute force takes $\Omega(2^n)$ time. **Exponential** : (

Time Complexity of Backtracking

```
def knapsack(items, bag_size):  
    # choose item arbitrarily from those that fit in bag  
    x = items.arbitrary_item(fitting_in=bag_size)  
  
    # if None, it means there was no item that fit  
    if x is None:  
        return 0  
  
    items.remove(x)  
    best_with = x.price + knapsack(items, bag_size - x.size)  
    best_without = knapsack(items, bag_size)  
    items.replace(x)  
  
    return max(best_with, best_without)
```

$T(n) =$

Backtracking Takes **Exponential Time**

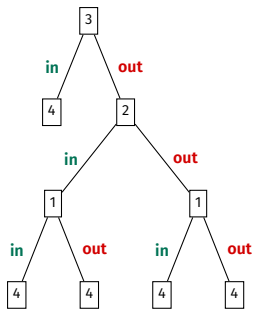
- ▶ ...in the worst case.
- ▶ This is just as bad as **brute force**.
- ▶ So why use it?
- ▶ Its worst case isn't always indicative of its practical performance.

Intuition

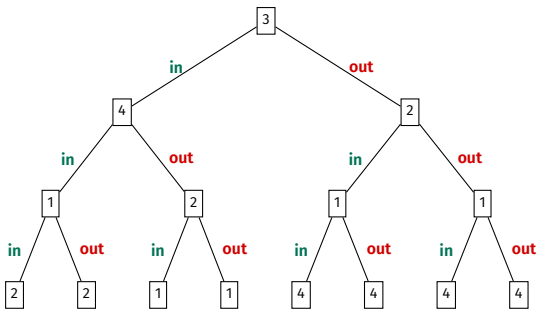
- ▶ Brute force tries all **possible** combinations.
 - ▶ E.g., all combinations of items, even if they don't fit in the bag.
- ▶ Backtracking tries all **valid** combinations.
 - ▶ E.g., all combinations of items that will fit in the bag.
- ▶ The number of valid combinations can be **much less** than the number of possible combinations.¹

¹Not always true!

Pruning



backtracking



brute force

Pruning

- ▶ Backtracking **prunes** branches that lead to invalid solutions.

Example

- ▶ 23 items with size/price chosen from $\text{Unif}([23, \dots, 46])$
- ▶ Bag size is 46
- ▶ Brute force: ?
- ▶ Backtracking: ?

Example

- ▶ 23 items with size/price chosen from $\text{Unif}([23, \dots, 46])$
- ▶ Bag size is 46
- ▶ Brute force: 52 seconds.
- ▶ Backtracking: ?

Example

- ▶ 23 items with size/price chosen from $\text{Unif}([23, \dots, 46])$
- ▶ Bag size is 46
- ▶ Brute force: 52 seconds.
- ▶ Backtracking: 4 milliseconds.

Example

- ▶ 300 items with size/price chosen from $\text{Unif}([150, \dots, 300])$
- ▶ Bag size is 600
- ▶ Brute force: ?
- ▶ Backtracking: ?

Example

- ▶ 300 items with size/price chosen from $\text{Unif}([150, \dots, 300])$
- ▶ Bag size is 600
- ▶ Brute force: $\approx 4.6 \times 10^{77}$ years
- ▶ Backtracking: ?

Example

- ▶ 300 items with size/price chosen from $\text{Unif}([150, \dots, 300])$
- ▶ Bag size is 600
- ▶ Brute force: $\approx 4.6 \times 10^{77}$ years
- ▶ Backtracking: 30 seconds.

Exercise

What is the **worst possible situation** for backtracking?
That is, when can we **not** prune any branches?

Backtracking Worst Case

- ▶ knapsack's **worst case** is when **bag size is very large**.
- ▶ All solutions are valid, aren't pruned.
- ▶ But this is actually an easy case!

Exercise

What is the optimal solution when the bag is very large (i.e., can fit everything)?

```
def knapsack_2(items, bag_size):  
    if sum(item.size for item in items) < bag_size:  
        return sum(item.price for item in items)  
  
    x = items.arbitrary_item(fitting_in=bag_size)  
  
    if x is None:  
        return 0  
  
    items.remove(item)  
    best_with = x.price + knapsack_2(items, bag_size - x.size)  
    best_without = knapsack_2(items, bag_size)  
    items.replace(x)  
  
    return max(best_with, best_without)
```

Pruning

- ▶ This further prunes the tree, resulting in speedup for some inputs.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 10 | Part 5

Branch and Bound

Example

- ▶ Suppose you have a bag of size 100.
- ▶ One of the items is a diamond.
 - ▶ Price: \$10,000. Size: 1
- ▶ The other 49 items are coal.
 - ▶ Price: \$1. Size: 1
- ▶ Do you even consider not taking the diamond?

Idea

1. Assume we take the diamond, compute best result.
 2. Find quick upper bound for not taking diamond.
 3. If upper bound is less than best for diamond, don't go down that branch.
- This is **branch and bound**; another way to prune tree.

Branch and Bound

```
def knapsack_bb(items, bag_size, find_upper_bound):  
    # try to make a good first choice  
    x = items.item_with_highest_price_density(fitting_in=bag_size)  
  
    if x is None:  
        return 0  
  
    items.remove(item)  
    best_with = x.price + knapsack_bb(items, bag_size - x.size)  
  
    upper_bound_without = find_upper_bound(items, bag_size)  
    if upper_bound_without > best_with:  
        # we have to look down the other branch...  
        best_without = knapsack_bb(items, bag_size)  
    else:  
        # prune that branch; don't look down it  
        best_without = 0  
  
    items.replace(x)  
  
    return max(best_with, best_without)
```

A Good First Choice

- ▶ Before, the first choice didn't affect efficiency.
 - ▶ We still explored all valid options.
- ▶ Now, it does.
 - ▶ A good first choice allows us to prune more branches.

Example

item	size (L)	price
1	50	\$40
2	25	\$25
3	95	\$1000
4	5	\$10

Upper Bounds for Knapsack

- ▶ How do we get a good upper bound?
- ▶ One idea: the solution to the *fractional* knapsack problem upper bounds that for 0/1 knapsack.

CS-GY 6033
Design and Analysis of Algorithms I

Lecture 10 | Part 6

Summary

Summary

- ▶ A backtracking approach is **guaranteed** to find an optimal answer.
- ▶ It is typically faster than brute force, but can still take **exponential time**.

Generalization

- ▶ Backtracking works for a **very wide range** of discrete optimization problems.
- ▶ Generalizes beyond “include or exclude” binary decision trees.
 - ▶ Any situation where you have a set of choices, and you can only pick one.

Summary

- ▶ We can speed up backtracking by pruning:
- ▶ Three ways to prune:
 1. Prune invalid branches (default).
 2. Prune “easy” cases.
 3. Prune by branching and bounding.

Summary

- ▶ Next: **dynamic programming**.
- ▶ We'll see it is “just” backtracking + a cache.

Where are we?

- ▶ We've been studying algorithm design.
- ▶ **Greedy algorithms**
 - ▶ Typically fast
 - ▶ But only guaranteed to find optimal answer for a select few problems (e.g., activity scheduling)
- ▶ **Backtracking**
 - ▶ Usually have bad worst case (exponential!)
 - ▶ But are guaranteed to find optimal answer.

Today

- ▶ **Dynamic Programming**: backtracking + memoization.
- ▶ Just as general as backtracking.
- ▶ And for some problems, **massively faster**.
- ▶ A “sledgehammer” of algorithm design.²

²Dasgupta, Papadimitriou, Vazirani

Today

- ▶ A new problem: weighted activity scheduling.
- ▶ We'll design a **dynamic programming** solution in steps:
 1. Backtracking solution.
 2. "Nicer" backtracking with repeating subproblems.
 3. Give backtracking algorithm a short-term memory.
- ▶ We'll turn an **exponential** time algorithm to **linear** by adding 2 lines of code.

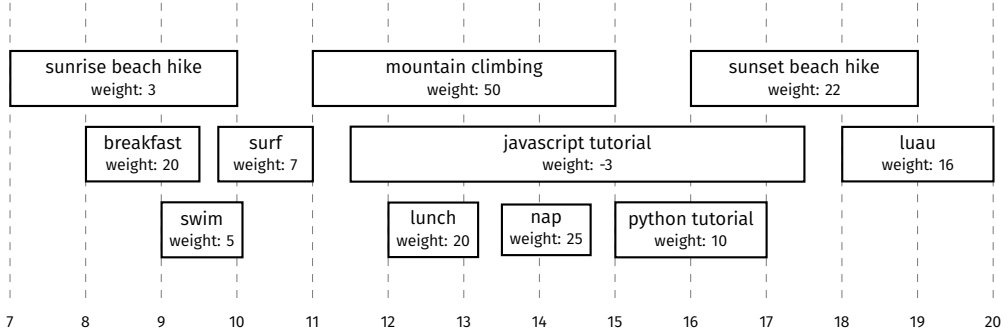
CS-GY 6033

Design and Analysis of Algorithms I

Lecture 10 | Part 7

Weighted Activity Selection Problem

Vacation Planning

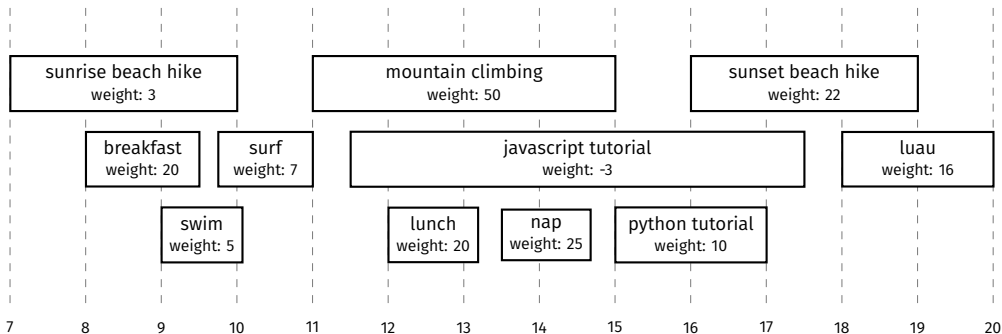


Weighted Activity Selection Problem

- ▶ **Given:** a set of activities each with start, finish, weight.
- ▶ **Goal:** Choose set of compatible activities so as to maximize total weight.

Exercise

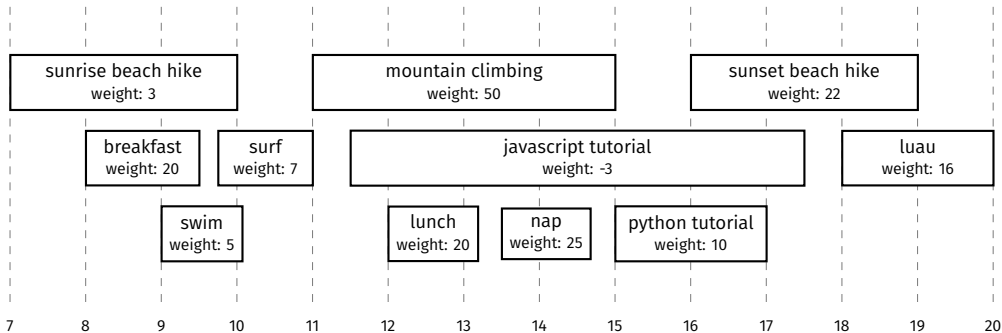
Find a schedule with the maximum total weight.



Greedy?

- ▶ In HW04 *unweighted* problem: maximize total number of activities.
- ▶ Greedy solution: take compatible activity that finishes earliest, repeat.
- ▶ This is **guaranteed** to find optimal in that problem.
- ▶ It **may not** find optimal for weighted problem.

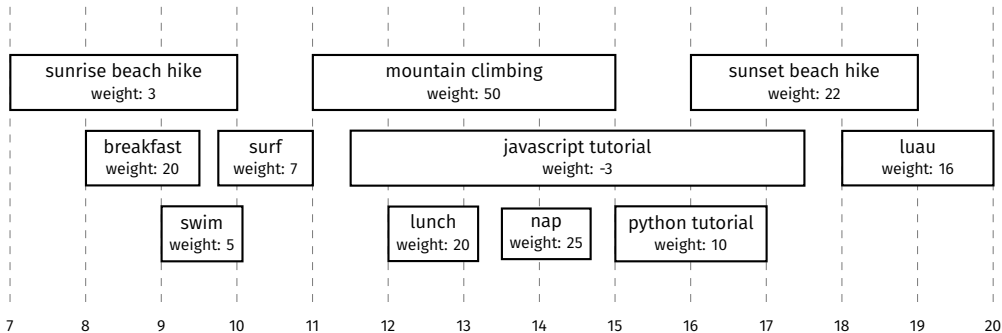
Greedy?



Greedy?

- ▶ Maybe a different greedy approach works?
- ▶ Idea: take compatible activity with **largest weight**.

Greedy?



Don't be greedy!

- ▶ The greedy approach is **not guaranteed** to find best.
- ▶ Note: you might get lucky on a particular instance!

What now?

- ▶ We'll try **backtracking**.
- ▶ It will take **exponential time**.
- ▶ But with a small change, we'll get a **linear time** algorithm that is guaranteed to find the best!

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 10 | Part 8

Step 01: Backtracking Solution

Backtracking

- ▶ We'll build up a schedule, one activity at a time.
- ▶ Choose an arbitrary activity, x.
 - ▶ Recursively see what happens if we **do** include x.
 - ▶ Recursively see what happens if we **don't** include x.
- ▶ This will try **all valid schedules**, keep the best.

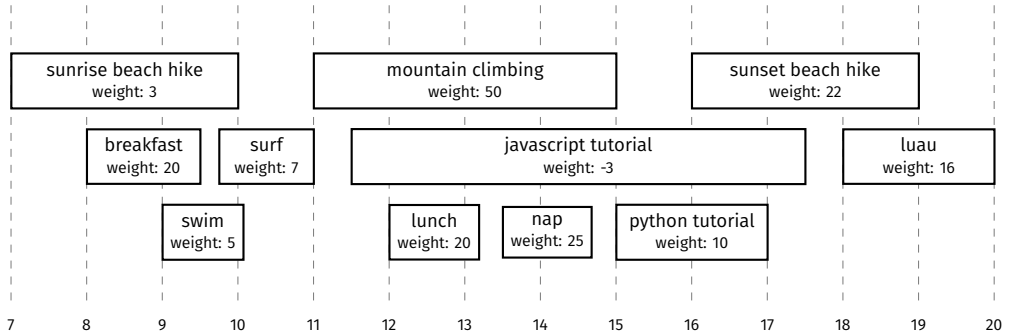
Backtracking

```
def mwsched_bt(activities):  
    if not activities:  
        return 0  
  
    # choose arbitrary activity  
    x = activities.choose_arbitrary()  
  
    # best with x  
    best_with = ...  
  
    # best without x  
    best_without = ...  
  
    return max(best_with, best_without)
```

Recursive Subproblems

- ▶ What is `BEST(activities)` if we assume that `x` **is** in schedule?
- ▶ Imagine choosing `x`.
 - ▶ Your current total weight is `x.weight`.
 - ▶ Activities left to choose from: those **compatible** with `x`.
- ▶ Clearly, you want the best outcome for *new* situation (subproblem).
- ▶ Answer: `x.weight + BEST(activities.compatible_with(x))`

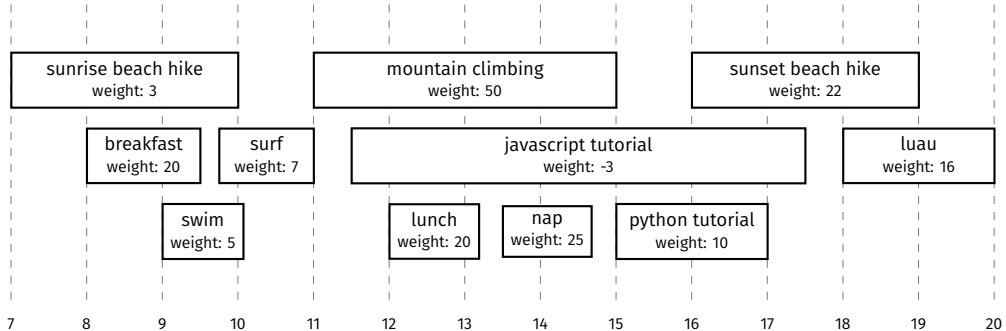
activities.compatible_with(x)



Recursive Subproblems

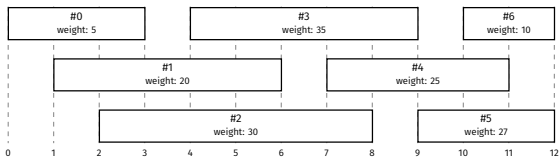
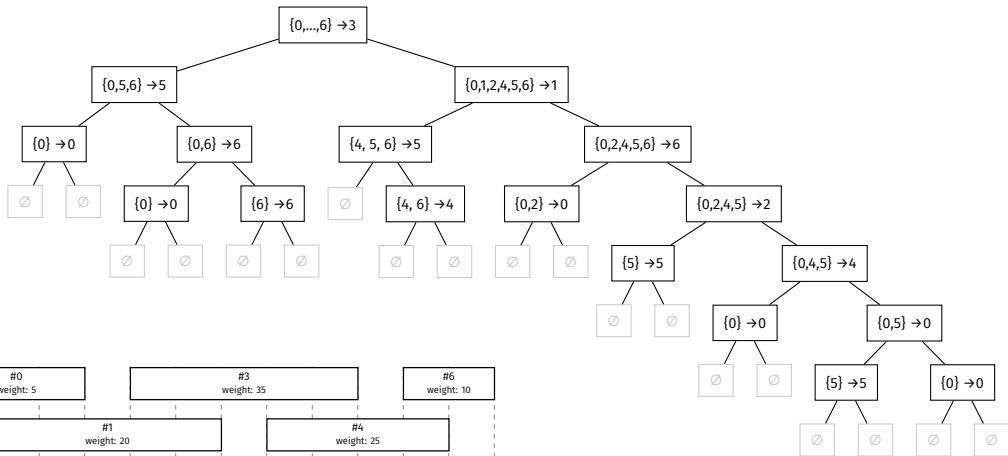
- ▶ What is `BEST(activities)` if we assume that `x` **is not** in schedule?
- ▶ Imagine not choosing `x`.
 - ▶ Your current total weight is $\ominus$.
 - ▶ Activities left to choose from: all except `x`.
- ▶ Clearly, you want the best outcome for *new* situation (subproblem).
- ▶ Answer: `BEST(activities.without(x))`

activities.without(x)



Backtracking

```
def mwsched_bt(activities):  
    if not activities:  
        return 0  
  
    # choose arbitrary activity  
    x = activities.choose_arbitrary()  
  
    # best with x  
    best_with = x.weight + mwsched_bt(activities.compatible_with(x))  
  
    # best without x  
    best_without = mwsched_bt(activities.without(x))  
  
    return max(best_with, best_without)
```



Efficiency

- ▶ Worst case: recursive calls on problem of size $n - 1$.
- ▶ Recurrence of form $T(n) = 2T(n - 1) + \Theta(\dots)$
- ▶ **Exponential time** in worst case.
- ▶ Could prune, branch & bound, but there's a better way.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 10 | Part 9

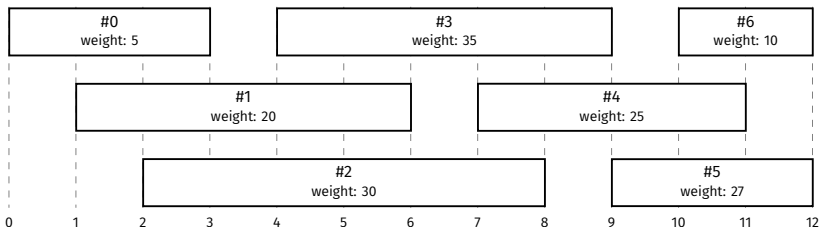
Step 02: A Nicer Backtracking Solution

Arbitrary Choices

- ▶ Our subproblems are arbitrary sets of activities.
 - ▶ E.g., {1, 3, 4, 5, 8, 11, 12}
- ▶ **Now:** If we make choice of next event more carefully, the subproblems look much nicer.
- ▶ Something great happens!

A Nicer Choice

- Instead of choosing arbitrarily, choose the activity that **starts first**.



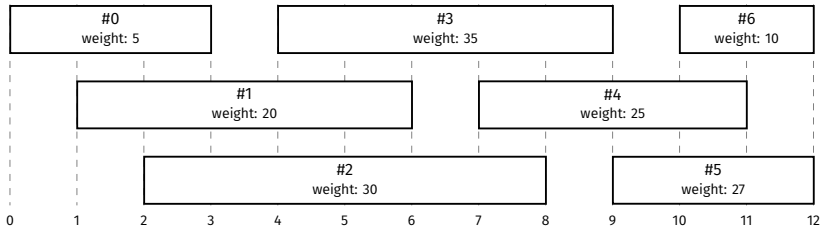
```
def mwsched_bt_nice(activities):  
    if not activities:  
        return 0  
  
    # choose activity which starts soonest  
    x = activities.starting_first()  
  
    # best with x  
    best_with = x.weight + mwsched_bt(activities.compatible_with(x))  
  
    # best without x  
    best_without = mwsched_bt(activities.without(x))  
  
    return max(best_with, best_without)
```

Assumption

- ▶ Assume:
 1. events are ordered by their **start time**.
 2. the event starting soonest is chosen.
- ▶ Then describing subproblems becomes easier.

activities.compatible_with(x)

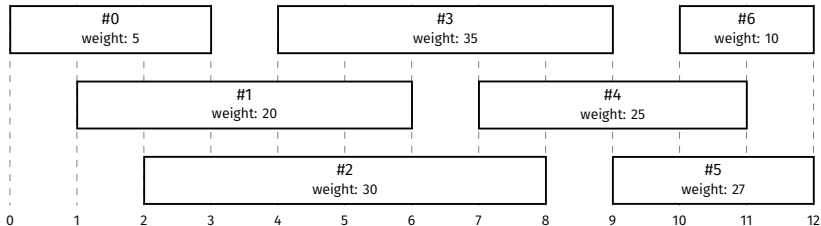
- Results in a “nice” set of the form $\{i, i + 1, \dots, n - 1\}$ ³



³Assuming x is the activity with first start time.

activities.without(x)

- Results in a “nice” set of the form $\{j, j + 1, \dots, n - 1\}$ ⁴



⁴Assuming x is the activity with first start time.

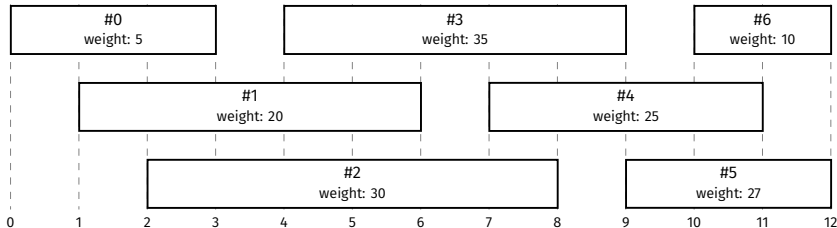
Representing Remaining Activities

- ▶ Assume events are in sorted order by start time.
- ▶ Subproblems are always of form $\{i, i + 1, i + 2, \dots, n - 1\}$
- ▶ We can specify them with a **single number**, i .

```
def mwsched_bt_nice(activities, first: int=0):  
    """Find best schedule using only events in activities[first:]  
    Assumes activities sorted by start time.  
    """  
    if first >= len(activities):  
        return 0  
  
    # choose first event  
    x = activities[first]  
  
    # best with x  
    next_compatible = index_of_next_compatible(activities, after=first)  
    best_with = x.weight + mwsched_bt_nice(activities, next_compatible)  
  
    # best without x  
    best_without = mwsched_bt_nice(activities, first + 1)  
  
    return max(best_with, best_without)
```

index_of_next_compatible()

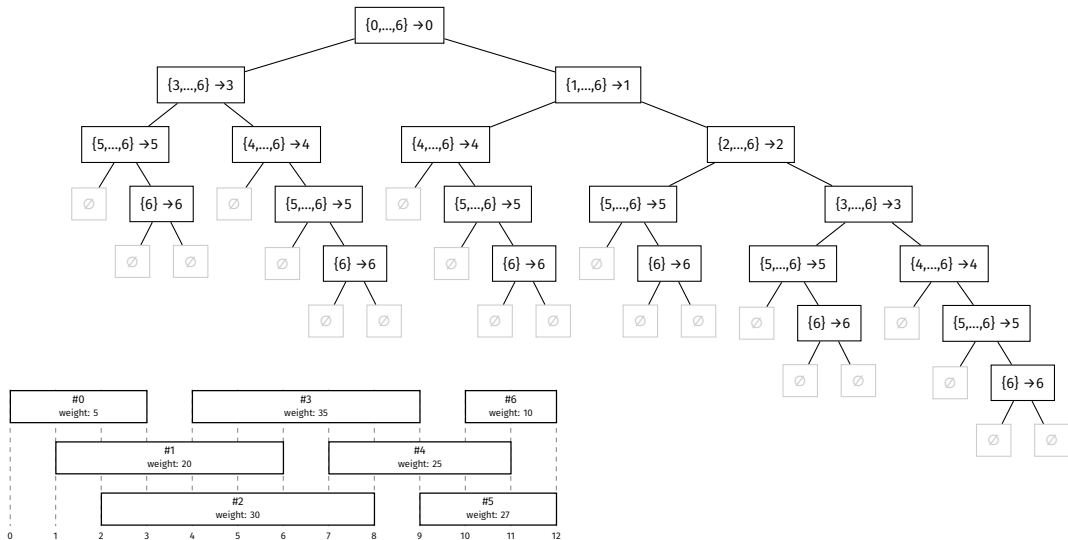
```
def index_of_next_compatible(activities, after: int):  
    """Find index of first event starting after `after` ends.  
    Assumes activities sorted by start time.  
    """  
    for j in range(after + 1, len(activities)):  
        if activities[j].start >= activities[after].finish:  
            return j  
    return len(activities)
```



What did we gain?

- ▶ Can specify subproblems with integers instead of sets.
 - ▶ Saves memory.
- ▶ But there's an **even better** consequence!

Overlapping Subproblems

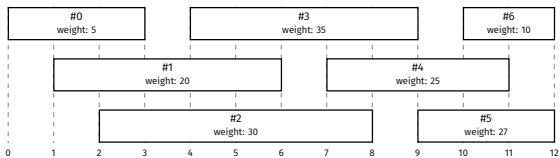
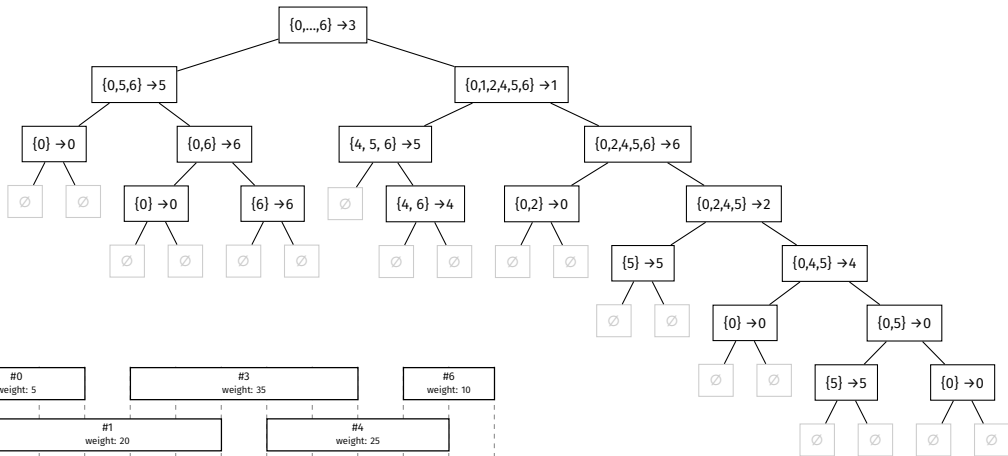


Overlapping Subproblems

- ▶ Backtracking doesn't have a memory.
- ▶ It will happily solve same subproblem over and over, getting same result each time.
- ▶ We'll speed it up by giving it a memory.

Important!

- ▶ Overlapping subproblems are a consequence of this more careful choice of event.
- ▶ When we chose arbitrarily, we **didn't** have (as many) overlapping subproblems.



CS-GY 6033

Design and Analysis of Algorithms I

Lecture 10 | Part 10

Step 03: Memoization

Backtracking + Memoization

- ▶ By making careful choices, we've found a backtracking solution with many overlapping subproblems.
- ▶ Idea:
 - ▶ First time we see a subproblem, save the result!
 - ▶ When we see it again, recall the solution.
- ▶ This is called **memoization**⁵.

⁵Not “memorization”. That would make too much sense.

Memoization

- ▶ Keep a **cache**: dictionary or array mapping subproblems to solutions.
- ▶ Before solving a subproblem, check if already in cache.
- ▶ After solving a subproblem, save result in cache.

```

def mwsched_dp(activities, first: int=0, cache=None):
    """Find best schedule using events in activities[first:].
    Assumes activities sorted by start time."""

    if cache is None: # cache[i] is solution of activities[i:]
        cache = [None] * len(activities)

    if first >= len(activities):
        return 0

    # save some work if we've already computed this
    if cache[first] is not None:
        return cache[first]

    # choose first event
    x = activities[first]

    # best with x
    next_compatible = index_of_next_compatible(activities, after=first)
    best_with = x.weight + mwsched_dp(activities, next_compatible, cache=cache)

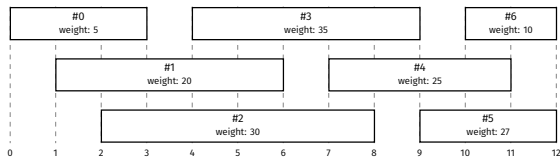
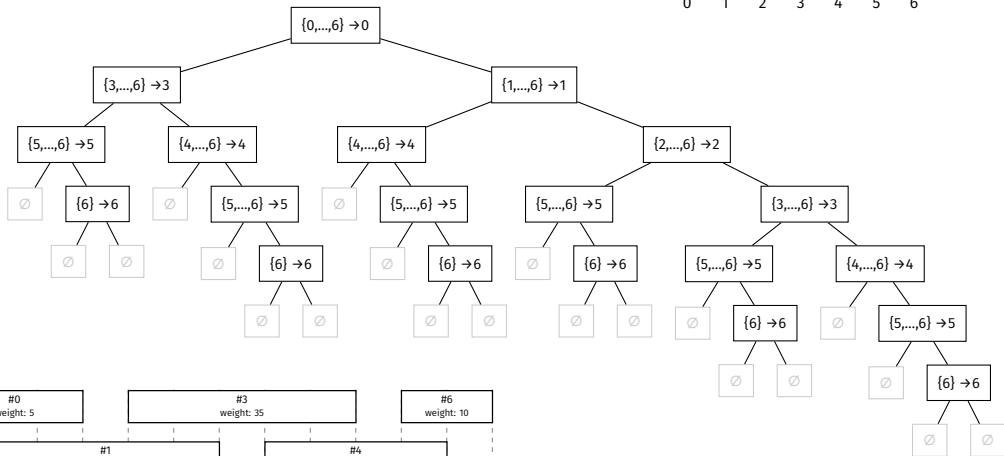
    # best without x
    best_without = mwsched_dp(activities, first + 1, cache=cache)

    best = max(best_with, best_without)

    # store result in cache for future reference
    cache[first] = best
    return best

```

cache = $\frac{\quad}{0} \frac{\quad}{1} \frac{\quad}{2} \frac{\quad}{3} \frac{\quad}{4} \frac{\quad}{5} \frac{\quad}{6}$



Time Complexity

- ▶ There are only n **unique** subproblems.
 - ▶ $\{0, \dots, n - 1\}, \{1, \dots, n - 1\}, \dots, \{n - 1\}$
- ▶ Solve each one once.
- ▶ The memoized solution takes $\Theta(n)$ time.

Dynamic Programming

- ▶ This approach (backtracking + memoization) is called “top-down” **dynamic programming**.
- ▶ Often reduces time from exponential to polynomial.

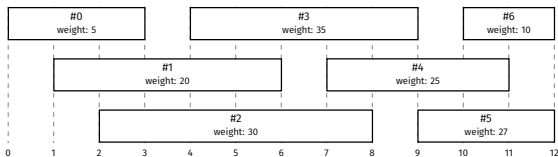
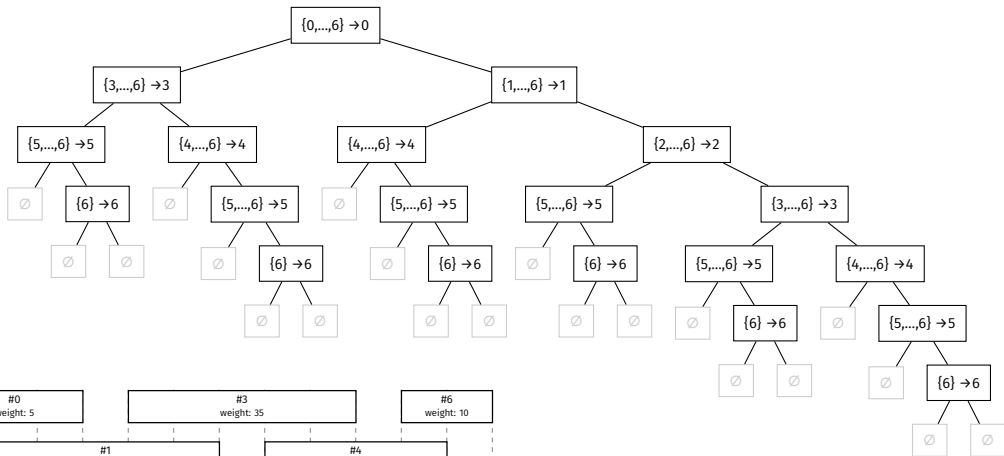
CS-GY 6033
Design and Analysis of Algorithms I

Lecture 10 | Part 11

Top-Down vs. Bottom-Up

Top-Down

- ▶ Backtracking + memoization is known as “top down” dynamic programming.
- ▶ We start at top level problem, recursively find subproblems.
- ▶ But we can start from bottom-level problems, too.



Bottom-Up

- ▶ The top-down recursive code solves problems in order:
 - ▶ $\{6\}, \{5, 6\}, \{4, \dots, 6\}, \{3, \dots, 6\}, \{2, \dots, 6\}, \{1, \dots, 6\}, \{0, \dots, 6\}$
- ▶ The bottom-up approach starts with easiest subproblem, iteratively solves harder subproblems.
- ▶ Solve $\{6\}$. Use it to solve $\{5, 6\}$. Use this to solve $\{4, \dots, 6\}$, etc.

```

def mwsched_bottom_up(activities):
    """Assumes activities sorted by start time."""
    n = len(activities)

    # best[i] is the weight of the best possible schedule that can be formed
    # using activities[i:]. best[n] is a dummy value; it represents the "base case"
    # solution of zero. best[0] is solution to the full problem.
    best = [None] * (n + 1)
    best[n] = 0

    # solve easiest subproblem: when we have one event, activities[n-1]
    best[n-1] = activities[n-1].weight

    # iteratively solve subproblems from small to big,
    # using solutions of smaller problems in solving big
    for first in reversed(range(n-1)):
        x = activities[first]

        # best with
        next_compatible = index_of_next_compatible(activities, after=first)
        best_with = x.weight + best[next_compatible]

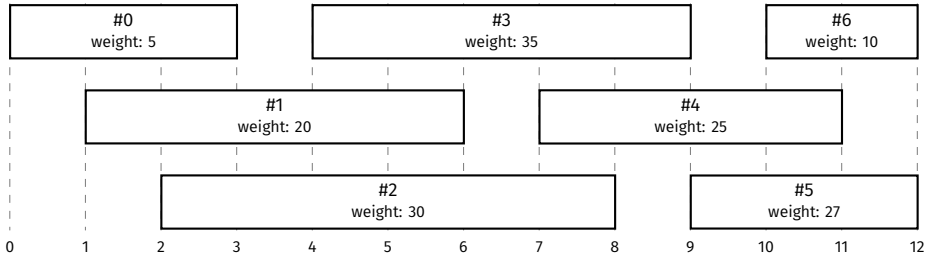
        # best without
        best_without = best[first + 1]

        best[first] = max(best_with, best_without)

    return best[0]

```

Example



best = $\frac{\quad}{0} \frac{\quad}{1} \frac{\quad}{2} \frac{\quad}{3} \frac{\quad}{4} \frac{\quad}{5} \frac{\quad}{6} \frac{\quad}{7}$

Which to use?

- ▶ Bottom-up and top-down will generally have same time complexity.
- ▶ Top-down arguably easier to design.
- ▶ Bottom-up avoids overhead of recursion.
- ▶ But bottom-up may solve unnecessary subproblems.

CS-GY 6033

Design and Analysis of Algorithms I

Lecture 10 | Part 12

Dynamic Programming

When can we use it?

- ▶ Memoization can be added to *any* backtracking algorithm.
- ▶ But it is only *useful* if there are **overlapping subproblems**.
- ▶ Not all problems yield overlapping subproblems.

How do we design them?

- ▶ General strategy for top-down:
 1. Write a backtracking solution.
 2. Modify backtracking solution to get overlapping subproblems that are “easy to describe”.⁶
 3. Add memoization.
- ▶ “Expert mode”: identify recursive substructure immediately.
- ▶ Can be tricky; need to be creative.

⁶Easier said than done.

How do we design them?

- ▶ General strategy for bottom-up:
 1. Write a top-down dynamic programming solution.
 2. Analyze the order in which cache is filled in.
 3. Iteratively solve subproblems in this order.

Are they guaranteed to be optimal?

- ▶ **Yes!** Dynamic programming *is* a form of backtracking, so it is guaranteed to find an optimal solution.

Is it at all useful for data science?

- ▶ **Yes!**
- ▶ Next time: the longest common subsequence problem and its applications to DNA string comparison.